

Guía Didáctica: Promesas en JavaScript

Apartado 1: Qué es una promesa en JavaScript

Explicación

Una **promesa** en JavaScript es un **objeto que representa la eventual finalización o fallo de una operación asíncrona**. Es decir, nos permite ejecutar tareas que tardan tiempo (como peticiones a un servidor) y luego manejar sus resultados cuando estén disponibles, sin bloquear la ejecución del resto del código.

Características principales:

- Representa **una operación pendiente, resuelta o rechazada**.
- Permite **manejar el resultado de operaciones asíncronas** de forma más clara que los callbacks.
- Se usa frecuentemente con **fetch API**, peticiones AJAX y otras operaciones asíncronas.

Estados de una promesa

Una promesa puede estar en **3 estados**:

1. **Pending (pendiente)** → La operación aún no se ha completado.
2. **Fulfilled (cumplida/resuelta)** → La operación se completó correctamente.
3. **Rejected (rechazada)** → La operación falló y produjo un error.

Métodos principales de una promesa

- `then()` → Se ejecuta cuando la promesa se resuelve correctamente.
- `catch()` → Se ejecuta cuando la promesa es rechazada.
- `finally()` → Se ejecuta siempre, haya éxito o error.

Ejemplo práctico 1: Promesa básica con éxito

Objetivo: Mostrar cómo funciona una promesa que se resuelve correctamente.

HTML + Bootstrap + JS

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
```

```

    <title>Promesa Básica</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">
    <h1>Ejemplo 1: Promesa Básica</h1>
    <button id="btnEj1" class="btn btn-primary">Ejecutar
Promesa</button>
    <div id="resultadoEj1" class="mt-3"></div>

    <script>
        const btnEj1 = document.getElementById('btnEj1');
        const resultadoEj1 = document.getElementById('resultadoEj1');

        btnEj1.addEventListener('click', () => {
            const promesa = new Promise((resolve, reject) => {
                setTimeout(() => {
                    resolve("La promesa se ha cumplido
correctamente!");
                }, 2000); // simula operación asíncrona
            });

            promesa.then((mensaje) => {
                resultadoEj1.innerHTML = `<div class="alert alert-
success">${mensaje}</div>`;
            });
        });
    </script>
</body>
</html>

```

Explicación:

- Se crea una promesa que se resuelve después de 2 segundos.
- `then()` captura el valor de éxito y lo muestra en pantalla.

Ejemplo práctico 2: Promesa con error (rechazada)

Objetivo: Mostrar cómo manejar errores en una promesa.

```

<h2>Ejemplo 2: Promesa con Error</h2>
<button id="btnEj2" class="btn btn-warning">Ejecutar Promesa con
Error</button>
<div id="resultadoEj2" class="mt-3"></div>

<script>
const btnEj2 = document.getElementById('btnEj2');
const resultadoEj2 = document.getElementById('resultadoEj2');

btnEj2.addEventListener('click', () => {
    const promesaError = new Promise((resolve, reject) => {
        setTimeout(() => {
            reject("La promesa ha fallado!");
        }, 2000);
    });
});

```

```

        promesaError
            .then(mensaje => {
                resultadoEj2.innerHTML = `<div class="alert alert-
success">${mensaje}</div>`;
            })
            .catch(error => {
                resultadoEj2.innerHTML = `<div class="alert alert-
danger">${error}</div>`;
            });
    });
</script>

```

Explicación:

- `reject()` indica fallo.
- `catch()` captura el error y lo muestra al usuario.

Ejemplo práctico 3: Promesa con `finally()`

```

<h2>Ejemplo 3: Promesa con finally</h2>
<button id="btnEj3" class="btn btn-info">Ejecutar Promesa</button>
<div id="resultadoEj3" class="mt-3"></div>

<script>
const btnEj3 = document.getElementById('btnEj3');
const resultadoEj3 = document.getElementById('resultadoEj3');

btnEj3.addEventListener('click', () => {
    const promesaFinally = new Promise((resolve, reject) => {
        const exito = Math.random() > 0.5;
        setTimeout(() => {
            exito ? resolve("Éxito!") : reject("Error!");
        }, 2000);
    });

    promesaFinally
        .then(msg => resultadoEj3.innerHTML = `<div class="alert
alert-success">${msg}</div>`)
        .catch(err => resultadoEj3.innerHTML = `<div class="alert
alert-danger">${err}</div>`)
        .finally(() => console.log("Operación terminada"));
    });
</script>

```

Explicación:

- `finally()` se ejecuta siempre, independientemente de si la promesa fue resuelta o rechazada.

Ejemplo práctico 4: Promesa que simula una petición al servidor

```

<h2>Ejemplo 4: Petición simulada a servidor</h2>
<button id="btnEj4" class="btn btn-success">Simular Petición</button>
<div id="resultadoEj4" class="mt-3"></div>

<script>
const btnEj4 = document.getElementById('btnEj4');
const resultadoEj4 = document.getElementById('resultadoEj4');

btnEj4.addEventListener('click', () => {
  function obtenerDatosServidor() {
    return new Promise((resolve, reject) => {
      setTimeout(() => {
        const exito = true;
        if(exito){
          resolve({id: 1, nombre: "Producto X"});
        } else {
          reject("No se pudieron obtener los datos del
servidor");
        }
      }, 2000);
    });
  }

  obtenerDatosServidor()
    .then(data => {
      resultadoEj4.innerHTML = `<div class="alert alert-
success">ID: ${data.id}, Nombre: ${data.nombre}</div>`;
    })
    .catch(err => {
      resultadoEj4.innerHTML = `<div class="alert alert-
danger">${err}</div>`;
    });
});
</script>

```

Explicación:

- Simula una petición asíncrona al servidor con `setTimeout`.
- Permite a los alumnos ver cómo manejar datos recibidos.

Ejemplo práctico 5: Promesas encadenadas (chaining)

```

<h2>Ejemplo 5: Promesas Encadenadas</h2>
<button id="btnEj5" class="btn btn-secondary">Ejecutar
Chaining</button>
<div id="resultadoEj5" class="mt-3"></div>

<script>
const btnEj5 = document.getElementById('btnEj5');
const resultadoEj5 = document.getElementById('resultadoEj5');

btnEj5.addEventListener('click', () => {
  new Promise((resolve, reject) => {
    setTimeout(() => resolve(2), 1000);
  })
  .then(num => {
    return num * 3;
  })
});

```

```

    })
    .then(num => {
        return num + 5;
    })
    .then(resultado => {
        resultadoEj5.innerHTML = `<div class="alert alert-
success">Resultado final: ${resultado}</div>`;
    })
    .catch(err => {
        resultadoEj5.innerHTML = `<div class="alert alert-
danger">${err}</div>`;
    });
});
</script>

```

Explicación:

- Cada `then()` recibe el valor del anterior y permite encadenar operaciones asíncronas.
- Muy útil para operaciones dependientes, como múltiples peticiones al servidor.

Ejercicios prácticos para alumnos (resueltos y propuestos)

Resueltos:

1. Crear una promesa que se cumpla después de 3 segundos y muestre un mensaje en un div.
2. Crear una promesa que se rechace si el número aleatorio es menor que 0.5.
3. Crear una promesa que simule recibir un array de productos del servidor y los muestre en una lista.
4. Encadenar promesas que tomen un número, lo multipliquen y luego le sumen un valor, mostrando el resultado final.

Propuestos (para que los alumnos hagan):

1. Crear una promesa que simule una **consulta de usuario a un servidor** y muestre su nombre y edad.
2. Crear una promesa que se cumpla o se rechace de manera **aleatoria**, mostrando un mensaje distinto según sea éxito o fallo.
3. Encadenar tres promesas que simulen **peticiones a distintas APIs**, mostrando los resultados uno tras otro.
4. Usar `finally()` para mostrar siempre un mensaje indicando que la operación terminó.